

```

In[1]:= (*Algorithm:LM-STLS (Levenberg-Marquardt Sequential Thresholded Least Squares)
Reference to cite when using this code:[1] A noise-corrected data-
driven approach for nonlinear dynamical system modeling in frequency domain,2026.
Code availability:This program is open for academic research
only.Unauthorized commercial redistribution is prohibited.
=====
=====*)

In[2]:= (*Import self-defined package for nonlinear sparse identification*)
Import["NCSINDyPackage.m", "Package"]

In[3]:= (* =====Basic System Configuration=====*)
(*n:nonlinear polynomial order;order:Fourier series expansion order*)
n = 7;
order = 5;
opts = {AccuracyGoal → 12, PrecisionGoal → 12, MaxStepFraction → 1/1000 };
(*Governing differential equation of nonlinear oscillator*)
Eq01 = {m x''[t] + c x'[t] +  $\sum_{j1=1}^n k_{j1} x[t]^{j1} - F \cos[\Omega t] == 0$ };

(*Initial conditions of dynamic system*)
InitCond01 = {x[0] == 0, x'[0] == 0};
(*True physical parameter values for simulation*)
Par01 =
Join[{m → 1, c → 0.4, F → 1}, Thread[Table[kj1, {j1, 1, n}] → {1, 0, 0.5, 0, 0.1, 0, 0}]];
(*List of undetermined physical coefficients to be identified*)
Var = Join[{m, c}, Table[kj1, {j1, 1, n}]];

```

```

In[8]:= (* =====
Fourier Series Model Definition=====*)
(*Displacement,velocity,acceleration Fourier series model*)
(*DispModel:displacement Fourier approximation*)
DispModel =  $A_{s,0} + \sum_{j1=1}^{order} (A_{s,j1} \cos[j1 \Omega t] + B_{s,j1} \sin[j1 \Omega t])$ ;
(*Coefficient list for displacement Fourier series*)
DispCoeff = Join[{As,0}, Flatten[Table[{As,j1, Bs,j1}, {j1, order}]]];

(*VeloModel:velocity Fourier approximation*)
VeloModel =  $A_{v,0} + \sum_{j1=1}^{order} (A_{v,j1} \cos[j1 \Omega t] + B_{v,j1} \sin[j1 \Omega t])$ ;
VeloCoeff = Join[{Av,0}, Flatten[Table[{Av,j1, Bv,j1}, {j1, order}]]];
(*AcceModel:acceleration Fourier approximation*)
AcceModel =  $A_{a,0} + \sum_{j1=1}^{order} (A_{a,j1} \cos[j1 \Omega t] + B_{a,j1} \sin[j1 \Omega t])$ ;
AcceCoeff = Join[{Aa,0}, Flatten[Table[{Aa,j1, Ba,j1}, {j1, order}]]];
(*Harmonic orders used for noise correction*)
OrderCorrect = {3, 4, 5};
(*Coefficient set of noise correction terms*)
CorrectModel = Sum[ $\delta A_{s,j1} \cos[j1 \Omega t] + \delta B_{s,j1} \sin[j1 \Omega t]$ , {j1, OrderCorrect}];
(*Coefficient set of noise correction terms*)
CorrectCoeff = DeleteCases[Flatten[Table[{ $\delta A_{s,j1}$ ,  $\delta B_{s,j1}$ }, {j1, OrderCorrect}]],  $\delta B_{s,0}$ ];
(*Sub-function:construct correction coefficients for each excitation frequency*)
CorrectCoeffj[jj_] :=
Join[{ $\delta A_{s,0}[jj]$ }, Flatten[Table[{ $\delta A_{s,j1}[jj]$ ,  $\delta B_{s,j1}[jj]$ }, {j1, order}]]] /.
x1_x2_x3_x4_ /; MemberQ[Complement[Range[0, order], OrderCorrect], x3] → 0;
(*Extract variable symbols of noise correction coefficients*)
CorrectCoeffjVar[jj_] :=
DeleteCases[Flatten[Table[{ $\delta A_{s,j1}[jj]$ ,  $\delta B_{s,j1}[jj]$ }, {j1, OrderCorrect}]],  $\delta B_{s,0}[jj]$ ];

In[17]:= (*Nonlinear stiffness model library constructed from displacement harmonics*)
StiffnessModel = ModelLibFun[{DispCoeff}, n, order];
(*Excitation frequency list for sweeping analysis*)
OmegaList = Range[0.4, 2.8, 0.4];
(*Sampling time sequence*)
TimeData = Range[800, 1000, 0.01];
(*Signal-to-Noise Ratio*) SNR = 10;

In[21]:= (*Load preset noise data from external file for reproducibility*)
Noise = Transpose[Import["C:/Users/Administrator/Desktop/Noise41.xls"][[1]]];

```

```

In[22]:= (* =====Generate Dynamic Response Data=====*)
For[j2 = 1, j2 ≤ Length[OmegaList], j2++,
  (*Solve ODE to obtain true displacement response*)
  Sol01[j2] = NDSolveValue[
    Join[Eq01, InitCond01] /. Par01 /. Ω → OmegaList[[j2]], x, {t, 0, 1000}, opts];
  (*True displacement time series data*)
  DispData[j2] =
    Array[{TimeData[[#]], Sol01[j2][TimeData[[#]]]} &, Length[TimeData]];
  DispSigTrue[j2] = Array[Sol01[j2][TimeData[[#]]] &, Length[TimeData]];
  (*Add preset noise to obtain measured noisy displacement*)
  (*DispSigNois[j2]=AddNoiseFun[DispSigTrue[j2],0,SNR];*)
  DispSigNois[j2] = Noise[[j2]];
  DispDataMeas[j2] = Array[
    {TimeData[[#]], DispSigTrue[j2][[#]] + DispSigNois[j2][[#]]} &, Length[TimeData]];
  (* =====
    Fourier Coefficient Identification via Fitting=====*)
  (*Fourier coefficients fitted from noisy measured data*)
  Ameas[x][j2] = DispCoeff /. FindFit[DispDataMeas[j2],
    DispModel /. Par01 /. Ω → OmegaList[[j2]], DispCoeff, t];
  (*Fourier coefficients fitted from noise-free true data*)
  Atrue[x][j2] = DispCoeff /.
    FindFit[DispData[j2], DispModel /. Par01 /. Ω → OmegaList[[j2]], DispCoeff, t];
  (*Fourier coefficients of pure noise component*)
  Anois[x][j2] = CorrectCoeff /.
    FindFit[Array[{TimeData[[#]], DispSigNois[j2][[#]]} &, Length[TimeData]],
    CorrectModel /. Par01 /. Ω → OmegaList[[j2]], CorrectCoeff, t];
  (*Noise correction for displacement harmonic coefficients*)
  A[x][j2] =
    ReplacePart[Ameas[x][j2] - (CorrectCoeffj[j2] /. Ω → OmegaList[[j2]]), 1 → 0];
  (* =====
    Full-State Reconstruction via SVA Transform=====*)
  (*Reconstruct velocity and acceleration from corrected displacement harmonics*)
  A[x'] [j2] = ReplacePart[SVATransform[A[x][j2],
    "rad/s", OmegaList[[j2]], "s2v"], 1 → 0];
  A[x''] [j2] = ReplacePart[SVATransform[A[x][j2], "rad/s",
    OmegaList[[j2]], "s2a"], 1 → 0];
  (*Excitation force harmonic coefficients*)
  A[f][j2] = Join[{0, F}, Table[0, 2 order - 1]] /. Par01;
  (*Construct nonlinear library matrix in frequency domain*)
  ModelLibrary[j2] = StiffnessModel /. Thread[DispCoeff → A[x][j2]];
  (*Assemble Jacobian-like matrix and residual vector for nonlinear least squares*)
  AA1[j2] = Join[{A[x''] [j2]}^T, {A[x'] [j2]}^T, ModelLibrary[j2], 2].{Var}^T // Chop;
  RR1[j2] = Flatten[AA1[j2] - {A[f][j2]}^T];
  Print[j2];
];

```

1

2

3

4

5

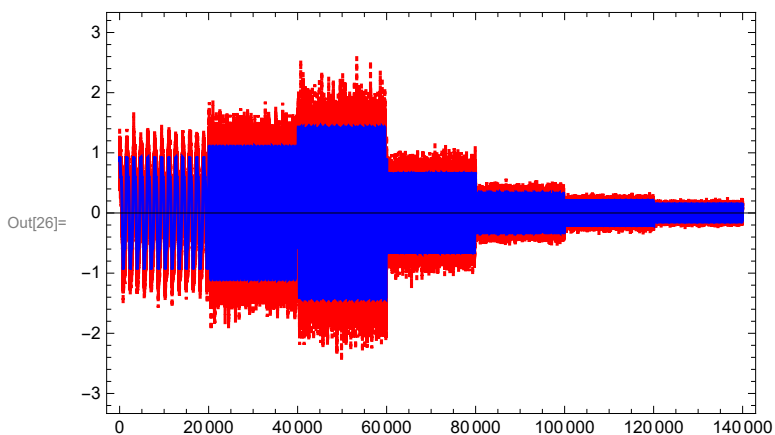
6

7

```

In[23]:= (* =====
Prepare Training Data for Sparse Regression=====*)
(*Collect noisy and noise-free training time-series data*)
TrainDataDispMeas = Table[DispDataMeas[j1], {j1, 1, Length[OmegaList]}];
TrainDataDispTrue = Table[DispData[j1], {j1, 1, Length[OmegaList]}];
(*Plot noisy and true displacement responses*)
Fig2aNoise =
  ListLinePlot[Flatten[Table[DispDataMeas[j1][[All, 2]], {j1, 1, Length[OmegaList]}]],
    PlotStyle -> {Red, Dotted}, Frame -> True];
Fig2aTrue = ListLinePlot[Flatten[Table[DispData[j1][[All, 2]],
  {j1, 1, Length[OmegaList]}]], PlotStyle -> {Blue}, Frame -> True];
Fig2a = Show[Fig2aNoise, Fig2aTrue, PlotRange -> {-3, 3}]

```



```

In[27]:= (*Combine residual across all excitation frequencies*)
Residual = Join@@ Table[RR1[j2], {j2, 1, Length[OmegaList]}];
(*Merge physical parameters and noise
correction coefficients into full variable set*)
Variable = Join[Var,
  Table[Join[CorrectCoeffjVar[j2], {j2, 1, Length[OmegaList]}] // Flatten];
(*Compute Jacobian matrix of residual with respect to unknown variables*)
JacobiFun = D[Residual, {Variable, 1}];
(* =====
LM-STLS Algorithm Hyperparameters=====*)
ConvTol = 10-6; maxLMIter = 100;
Lambda = 0.02; maxSparseIter = 10;
(*Indices of physical terms forced to be retained during thresholding*)
InitbigIndices = Join[Range[Length[Var] + 1, Length[Variable]]];

In[33]:= (*Initial guess of unknown parameter vector*)
InitGuess = ReplacePart[ReplacePart[0.2 * Variable /. Par01,
  Thread[Range[Length[Var] + 1, Length[Variable]] -> 0]], Thread[{4, 6, 8, 9} -> 1]];
(*Execute proposed LM-STLS nonlinear sparse regression*)
Xi = NonLinearSparsifyDynamics[JacobiFun, Residual, Variable,
  InitGuess, ConvTol, maxLMIter, Lambda, maxSparseIter, InitbigIndices];

```

$$\begin{array}{l} \{1, 49\} \\ \{2, 47\} \\ \{3, 47\} \\ \{4, 47\} \\ \{5, 47\} \\ \{6, 47\} \\ \{7, 47\} \\ \{8, 47\} \\ \{9, 47\} \\ \{10, 47\} \end{array}$$

```
In[35]:= (* =====
Quantitative Evaluation Metrics=====)
(*g1: Number of non-zero identified terms (sparsity evaluation)*)
(*g2: Relative L2 error between identified
parameters and true values (accuracy evaluation)*)
g1 = Table[Count[Xi[[1]][[j1, 1 ;; 9]], x_ /; x ≠ 0], {j1, 1, Xi[[2]] // Total}];
g2 = Table[Norm[Xi[[1]][[j1, 1 ;; 9]] - Var /. Par01] / Norm[Var /. Par01],
{j1, 1, Xi[[2]] // Total}];
```

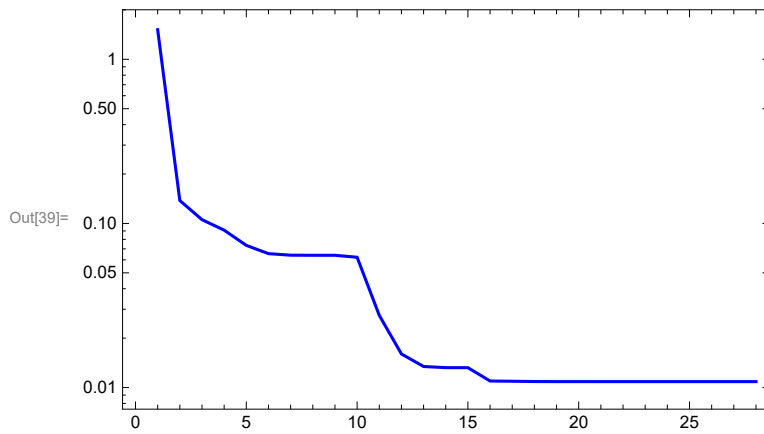
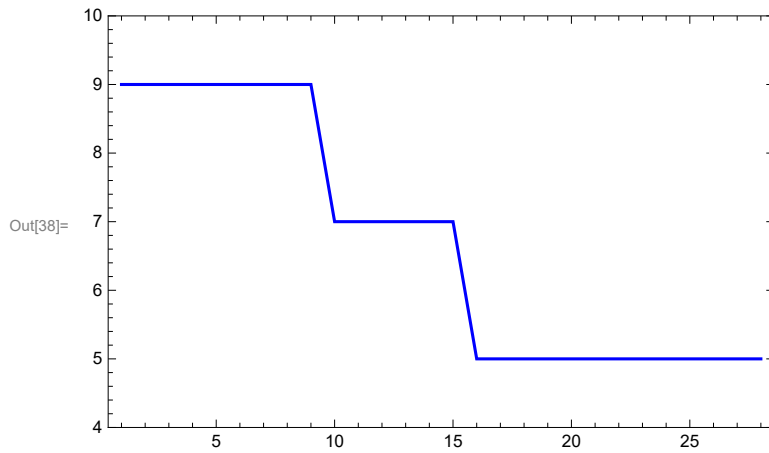
```
In[37]:= (*Show iterative evolution of parameter
          identification for {m,c,k1,k2,k3,k4,k5,k6,k7}*)
          Xi[[1]] // MatrixForm
```

[illegible]

```
In[38]:= (*Plot sparsity and error convergence curve*)
```

```
Fig2b = ListLinePlot[g1, PlotRange → {All, {4, 10}}, Frame → True, PlotStyle → Blue]
```

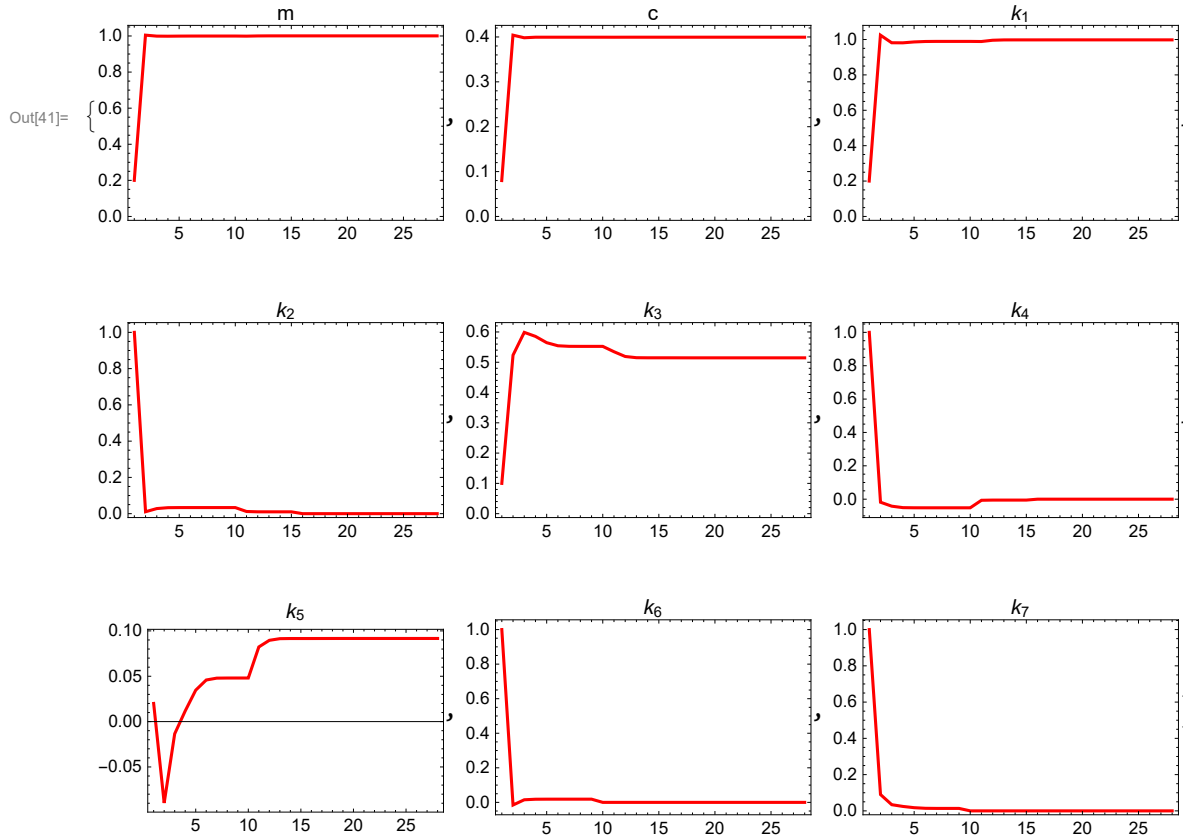
```
Fig2c = ListLogPlot[g2, Joined → True, Frame → True, PlotStyle → Blue]
```



```

In[40]:= (*Plot iterative convergence curve for each physical parameter m,c,k1~k7*)
plotlabel = {"m", "c", "k1", "k2", "k3", "k4", "k5", "k6", "k7"};
Fig3 = Table[
  ListLinePlot[Table[Xi[[1]][[j1]][[j2]], {j1, 1, Length[Xi[[1]]}], PlotStyle → Red,
    PlotRange → All, PlotLabel → plotlabel[[j2]], Frame → True], {j2, 1, 9}]

```



```

In[42]:= (*Custom vertical axis range for clearer visualization*)
YRange = {{0.997, 1.005}, {0.395, 0.405}, {0.96, 1.03}, {-0.01, 0.04},
          {0.5, 0.6}, {-0.06, 0.01}, {-0.1, 0.1}, {-0.03, 0.03}, {-0.01, 0.1}};
Fig3Local = Table[ListLinePlot[Table[Xi[[1]][[j1]][[j2]], {j1, 1, Length[Xi[[1]]]}],
  PlotStyle → Blue, PlotRange → YRange[[j2]],
  PlotLabel → plotlabel[[j2]], Frame → True], {j2, 1, 9}]

```

